GUIDE 57 · GOVERN

Answer a security questionnaire about data and cryptography

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=57-answer-a-security-questionnaire-about-data-and-cryptography. This copy was made on 4 November 2026.

Time: 30 minutes. **You'll end up with:** answers to the data-protection and cryptography sections of a vendor security questionnaire (SIG, CAIQ, or your customer's own) - each backed by what your QueryAPIGate deployment actually does, checked on it, not taken on trust.

Every command and output below was run against a real server: gunicorn with the image's configuration, encryption at rest on (`QUERYAPIGATE_SECRET_KEY`), and people signing in with JWTs (`QUERYAPIGATE_JWT_SECRET`), calling a saved query that takes a customer's email address. The reference throughout is Where data goes.

"What personal data does the system process, and why?"

From the page's personal data table: callers' names (an email address, for a person signed in with a token - `jwt:alice@example.com`), administrators' names and email addresses, IP addresses, an AI agent's write arguments - and whatever your databases hold, which passes through.

Answer it for *your* deployment by what you've turned on: no SSO and no JWTs, and no person is named at all, only API keys.

"Is personal data written to logs?"

A call that sends the email address in the URL:

```text
GET /q/orders?email=alice@example.com      Authorization: Bearer <alice's token>
[{"id": 1, "total": 42.5}, {"id": 3, "total": 7.25}]
```

What the access log and the application log say about it:

```text
21:25:26,945 INFO queryapigate [5c383d185428 key=jwt:alice@example.com]: Executing on shop (sqlite),
limit=10 offset=0
             timeout=30.0: SELECT id, total FROM orders WHERE email = :email
21:25:26,946 INFO queryapigate [5c383d185428 key=jwt:alice@example.com]: GET /q/orders → 200 in 2.8ms
127.0.0.1 - [12/Oct/2026:21:25:26 +0530] "GET /q/orders HTTP/1.1" 200 46 0.003582s rid=5c383d185428
"curl/8.5.0"
```

The path, never the query string; no referrer; the statement with its placeholder, `:email` - the value is bound, not written into it. The caller is named: `jwt:alice@example.com` is personal data, in the log. Lines in the log with the email address that was in the URL:

```text
0
```

The answer: *Requests are logged by path, method, status, time, request id, client address and caller name - a signed-in person's caller name is their identifier; parameter values are not logged* - unless you set `QUERYAPIGATE_ACCESS_LOG_QUERY=1`. Say what can still reach the log: an ad-hoc statement's text (`QUERYAPIGATE_HISTORY_ADHOC_SQL=hash` keeps it to a hash), and a database's own error message, which can quote a value.

"Is data encrypted at rest?"

A connection's password, as the store's file holds it:

```text
warehouse password, as stored: enc:gAAAAABqzQLfRV4CV5r8TOcz0G3Q0nJWxsBk_3tW...
```

Fernet - AES-128 and HMAC-SHA256 - under `QUERYAPIGATE_SECRET_KEY`, kept outside the store; API keys and admin tokens are only ever stored as SHA-256 hashes. The answer: *credentials at rest are encrypted or hashed; the store itself (configuration and run history, no query results) should sit on an encrypted volume or database, which is the platform's* - QueryAPIGate doesn't encrypt the whole file. Or keep passwords out of the store entirely: references to Vault or AWS Secrets Manager (ADR 0015).

"How long is each kind of data retained?"

From the server, as it's configured (`GET /api/v1/settings`):

```text
QUERYAPIGATE_HISTORY_LIMIT: 50 runs
QUERYAPIGATE_HISTORY_RETENTION_DAYS: per-version limit
QUERYAPIGATE_HISTORY_ADHOC_SQL: text
QUERYAPIGATE_AUDIT_LOG_LIMIT: 500
QUERYAPIGATE_ACCESS_LOG_QUERY: off
QUERYAPIGATE_SECRET_KEY: enabled
```

And what a run's history keeps:

```text
{'duration_ms': 1, 'key_name': 'jwt:bob@example.com', 'query': 'orders', 'rows': 1, 'status': 'success',
 'transport': 'rest', 'version': 1}
```

Who, which query, how many rows, how long - no parameter values, no rows. The page's store table gives every kind's retention and the setting that bounds it; copy the rows that apply. Logs, traces and SIEM copies are kept as your platforms keep them - say so, with their retention.

"Can you delete an individual's data on request?"

How much there is - `queryapigate history forget ... --dry-run`, or through the API:

```text
DELETE /api/v1/history?caller=jwt:alice@example.com&dry_run=true
{"runs": 3, "usage_periods": 0}
```

```shell
queryapigate history forget jwt:alice@example.com
```

```text
Deleted 3 run(s) and 0 usage period(s) of jwt:alice@example.com; recorded in the audit log by its hash,
not its name.
```

Alice's runs are gone; Bob's are not (`0 runs` and `1 run`). The audit log records that it happened:

```text
{'actor': 'cli', 'action': 'forget_caller', 'target': 'history',
 'changes': {'caller_sha256': '9e5bf797f99329d553c1ded1513172191bcaed59d54d26322c946aaf41b2e08d',
'runs': 3, ...}}
```

`sha256("jwt:alice@example.com")` is `9e5bf797...e08d`: anyone who knows the name can confirm the request was met; the log names no one to anyone who doesn't. What isn't erased, and why - the audit log's entries about administrators, write requests for their 30 days, your log platform - is on the page.

"Is the cryptography FIPS 140 validated?"

The precise answer: *QueryAPIGate uses only FIPS-approved algorithms, and runs on a FIPS 140-validated module when the platform provides one.* Validation belongs to the module (OpenSSL), not the application. The page lists every use - what it protects, the algorithm, the library - and a test fails if the code ever gains one that isn't approved.

On Red Hat Enterprise Linux 9 in FIPS mode, with Python and `cryptography` from the distribution - what CI runs the whole suite on:

```text
OpenSSL (ssl): OpenSSL 3.5.8 25 Aug 2026
cryptography 41.0.7 on OpenSSL 3.5.8 25 Aug 2026 - FIPS: True
md5 refused: [digital envelope routines] unsupported
Fernet round trip: True
```

Say what is outside that boundary in your deployment: the PostgreSQL and MySQL drivers' binary wheels (build `psycopg2` from source), DuckDB's TLS for remote files, and a JDBC or H2 connection's Java runtime.

"Who can access personal data?"

By capability, per administrator role: run history and the audit log need `observe`; keys need `access.read`; an agent's write arguments need `writes.approve`; erasing someone, `history.forget` - owners only. Callers see only what their key or role grants, after column masking. The page's tables have a "Read by" column for each kind of data.

Good to know

- **Keep the answers current by re-running the checks**, not by copying this page: the settings, a request with a value in the URL and a look at the log, `forget --dry-run`. A questionnaire answered from the running system stays right when the deployment changes.
- **A record of processing** (GDPR Article 30) can be built from the page's personal-data and store tables: the categories, the purposes, the retention, who reads it.
- **QueryAPIGate isn't certified, and can't be:** SOC 2 and ISO 27001 audit your organisation. The page and these checks are what your auditor reads about this part of it.

Next steps

- Where data goes - the whole inventory.
- Read the threat model.
- Use short-lived database credentials from Vault - no password in the store at all.